

§ 7.1 查找

7.1.1 循关键词访问

所谓的查找或搜索（**search**），指从一组数据对象中找出符合特定条件者，这是构建算法的一种基本而重要的操作。其中的数据对象，统一地表示和实现为词条（**entry**）的形式；不同词条之间，依照各自的关键词（**key**）彼此区分。根据身份证号查找特定公民，根据车牌号查找特定车辆，根据国际统一书号查找特定图书，均属于根据关键词查找特定词条的实例。

请注意，与此前的“循序访问”和“循位置访问”等完全不同，这一新的访问方式，与数据对象的物理位置或逻辑次序均无关。实际上，查找的过程与结果，仅仅取决于目标对象的关键词，故这种方式亦称作循关键词访问（**call-by-key**）。

7.1.2 词条

一般地，查找集内的元素，均可视作如代码7.1所示词条模板类**Entry**的实例化对象。

```
1 template <typename K, typename V> struct Entry { //词条模板类
2     K key; V value; //关键词、数值
3     Entry(K k = K(), V v = V()) : key(k), value(v) {}; //默认构造函数
4     Entry(Entry<K, V> const& e) : key(e.key), value(e.value) {}; //基于克隆的构造函数
5     bool operator<(Entry<K, V> const& e) { return key < e.key; } //比较器：小于
6     bool operator>(Entry<K, V> const& e) { return key > e.key; } //比较器：大于
7     bool operator==(Entry<K, V> const& e) { return key == e.key; } //判等器：等于
8     bool operator!=(Entry<K, V> const& e) { return key != e.key; } //判等器：不等于
9 }; //得益于比较器和判等器，从此往后，不必严格区分词条及其对应的关键词
```

代码7.1 词条模板类**Entry**

词条对象拥有成员变量**key**和**value**。前者作为特征，是词条之间比对和比较的依据；后者为实际的数据。若词条对应于商品的销售记录，则**key**为其条形码扫描码，**value**可以是其单价或库存量等信息。设置词条类只为保证查找算法接口的统一，故不必过度封装。

7.1.3 序与比较器

由代码7.1可见，通过重载对应的操作符，可将词条的判等与比较等操作转化为关键词的判等与比较（故在不致歧义时，往往无需严格区分词条及其关键词）。当然，这里隐含地做了一个假定——所有词条构成一个全序关系，可以相互比对和比较。需指出的是，这一假定条件不见得总是满足。比如在人事数据库中，作为姓名的关键词之间并不具有天然的大小次序。另外，在任务相对单纯但更加讲求效率的某些场合，并不允许花费过多时间来维护全序关系，只能转而付出有限的代价维护一个偏序关系。后者的一个实例，即第10章将要介绍的优先级队列——根据其ADT接口规范，只需高效地跟踪全局的极值元素，其它元素一般无需直接访问。

实际上，任意词条之间可相互比较大小，也是此前（2.6.5节至2.6.8节）有序向量得以定义，以及二分查找算法赖以成立的基本前提。以下将基于同样的前提，讨论如何将二分查找的技巧融入二叉树结构，进而借助二叉搜索树以实现高效的查找。

§ 7.2 二叉搜索树

7.2.1 顺序性

若二叉树中各节点所对应的词条之间支持大小比较，则在不致歧义的情况下，我们可以不必严格区分树中的节点、节点对应的词条以及词条内部所存的关键码。

如图7.2所示，在所谓的二叉搜索树（binary search tree）中，处处都满足顺序性：

任一节点 r 的左（右）子树中，所有节点（若存在）均不大于（不小于） r

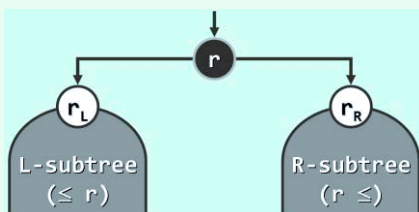


图7.2 二叉搜索树即处处满足顺序性的二叉树

为回避边界情况，这里不妨暂且假定所有节点互不相等。于是，上述顺序性便可简化表述为：

任一节点 r 的左（右）子树中，所有节点（若存在）均小于（大于） r

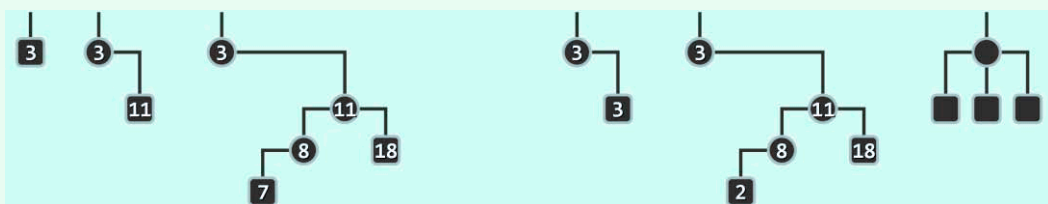


图7.3 二叉搜索树的三个实例（左），以及三个反例（右）

当然，在实际应用中，对相等元素的禁止既不自然也不必要。读者可在本书所给代码的基础上继续扩展，使得二叉搜索树的接口支持相等词条的同时并存（习题[7-10]）。比如，在去除掉这一限制之后，图7.3中原来的第一个反例，将转而成为合法的二叉搜索树。

7.2.2 中序遍历序列

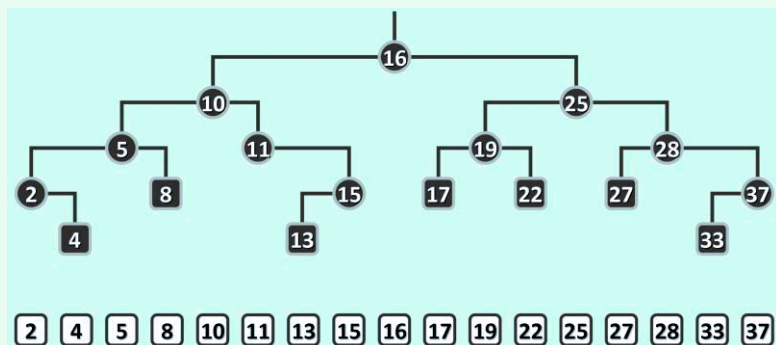


图7.4 二叉搜索树（上）的中序遍历序列（下），必然单调非降

顺序性是一项很强的条件。实际上，搜索树中节点之间的全序关系，已完全“蕴含”于这一